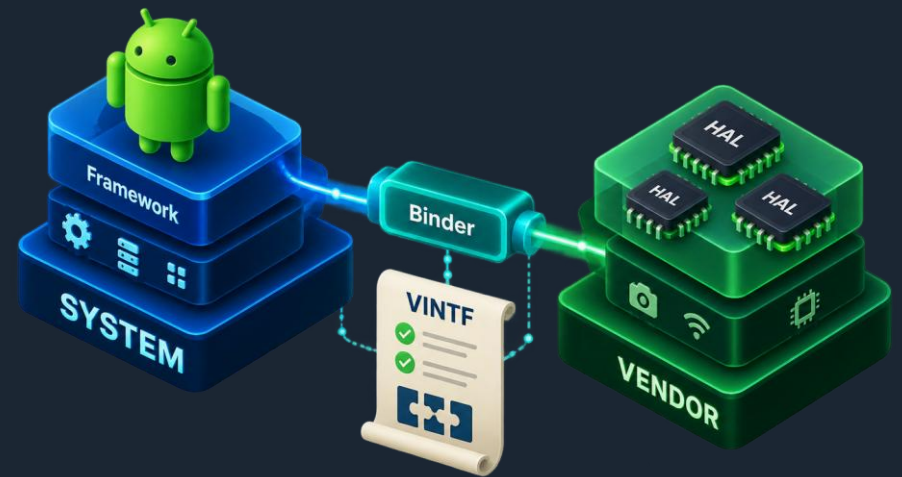


Android Framework 16 심화

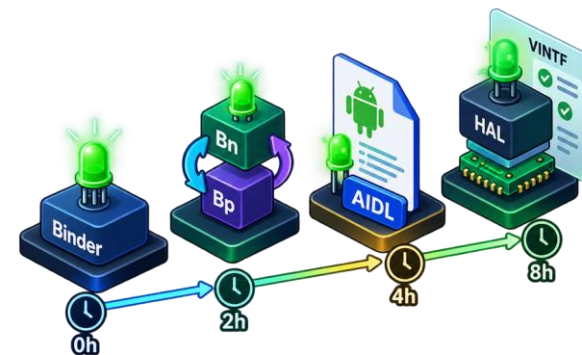
Day 2 — Native Binder Service · HAL · System Service

libbinder 수작업 Bn/Bp → AIDL cpp/ndk → UID 접근 제어 → init.rc · SELinux →
AIDL HAL · VINTF → Java System Service



오늘의 일정 — 8시간

Part	내용	소요
Part 0 환경·등록 실습	실습 1 환경 점검 · Android.bp 일괄 등록 · ctags/vim 소스 분석 환경	25분
Part 1 강의	Ch 1 libbinder 계층 · Ch 2 AIDL Native backend · Ch 3 System Service·SELinux · Ch 4 HAL·VINTF	130분
Part 2 실습	실습 2 ~ 11 (BBinder → 수작업 Bn/Bp → Parcel → Callback → AIDL → UID → init.rc/SELinux → HAL → SystemServer) · 실습 12 선택	305분
Part 3 데모 · 정리	실습 13 Binder 비용 측정·strace · Worksheet	35분



오늘의 축: 같은 LED 서비스를 raw BBinder → 수작업 Bn/Bp → AIDL 자동 생성 → UID 검사 → init.rc 데몬 → vendor HAL 로 발전시킨다

Part 0 · 실습 1 — 환경 점검 · Android.bp 일괄 등록 · ctags

1

빌드 환경 · Emulator 확인

lunch · \$OUT · adb root · getenforce = Permissive

2

day2/ 디렉토리 + 빈 소스 + Android.bp 한 파일

binder1~7 · auth · rc_service · led_hal 을 한 번에 등록

3

Soong 분석 1회 실행

m my_server_test_1 → analyzing Android.bp 통과 (링크 오류는 정상)

4

ctags · ~/.vimrc

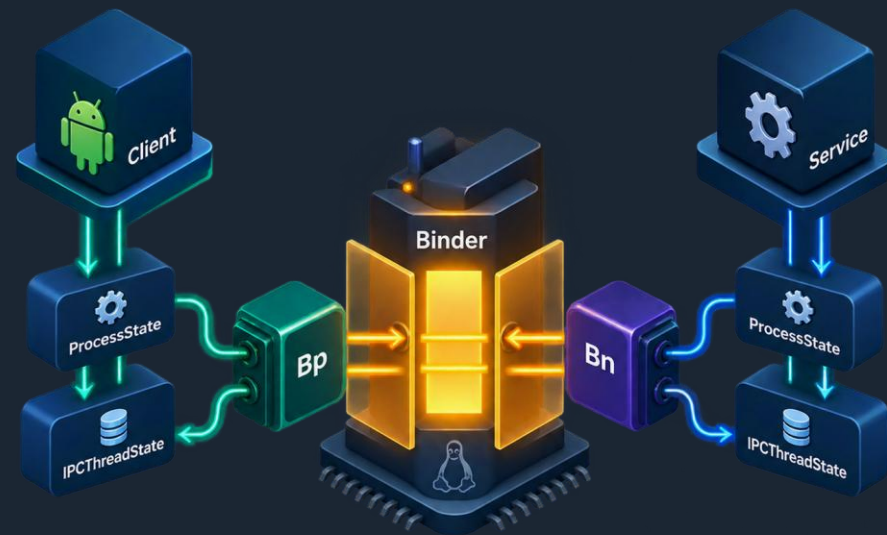
libbinder · servicemanager · day2 tags → :tag BBinder::onTransact



소요 25분 · .bp 는 오늘 이 한 번만 손댄다 — 이후 .cpp 만 편집 → mm 수신풀

Native Binder — libbinder 계층

45분 · ProcessState · IPCThreadState · BBinder/BpBinder · BnInterface/BpInterface · Parcel · 역방향 호출



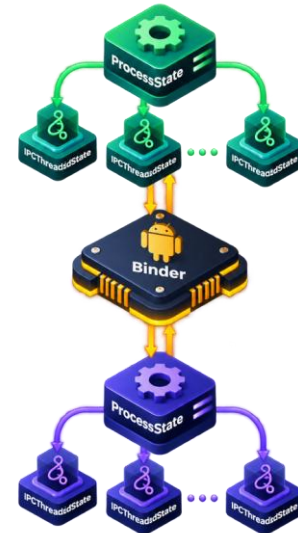
libbinder 의 두 축 — ProcessState 와 IPCThreadState

ProcessState — Process 당 1

- open("/dev/binder") + BINDER_VERSION
- mmap 1 MB — Transaction Buffer
- BINDER_SET_MAX_THREADS (15)
- startThreadPool() — Binder Thread 생성
- self() 최초 호출 시 생성

IPCThreadState — Thread 당 1

- transact() → ioctl(BINDER_WRITE_READ)
- joinThreadPool() — 요청 대기 루프
- getCallingUid() / getCallingPid()
- 처리 중 Transaction 의 발신자 보관
- TLS(thread-local) 에 저장



```
sp<ProcessState> proc(ProcessState::self()); // /dev/binder open + mmap
sp<IServiceManager> sm = defaultServiceManager(); // handle 0
sm->addService(String16("my.service"), new BBinder); // 등록
ProcessState::self()->startThreadPool(); // 추가 Binder Thread
IPCThreadState::self()->joinThreadPool(); // main 도 합류 - 반환 안 함
```

▶ 애니메이션 1 · libbinder 계층

객체 모델 — IBinder / IInterface 와 Bn / Bp

IBinder

IInterface

BBinder
로컬·onTransact

BpBinder
원격 handle

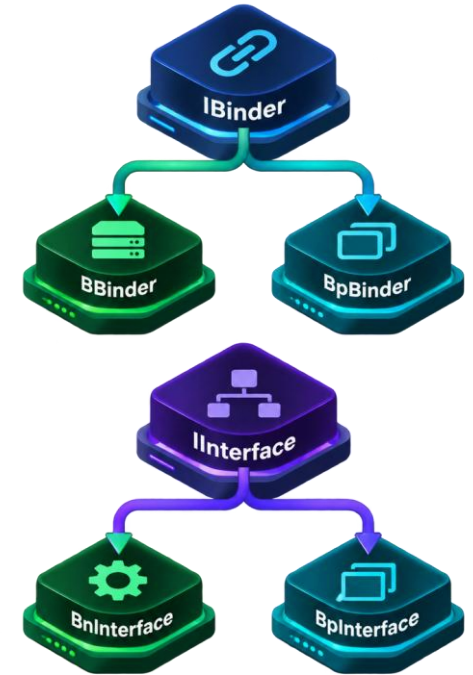
BnInterface<I>
= BBinder + I

BpInterface<I>
remote()→Bp

우리가 쓰는 클래스 (실습 3~4)	상속	역할
ILedService	IInterface	순수 가상 인터페이스 · DECLARE_META_INTERFACE
BnLedService	BnInterface<ILedService>	Stub · onTransact(code) → 가상 함수
BpLedService	BpInterface<ILedService>	Proxy · Parcel 직렬화 + remote()->transact()
LedService	BnLedService	구현체 · ledOn() 의 실제 일

Java (Day 1) 대응

- IBinder ↔ IBinder
- BBinder ↔ Binder
- BpBinder ↔ BinderProxy
- BnInterface ↔ I.Stub
- BpInterface ↔ I.Stub.Proxy
- interface_cast ↔ Stub.asInterface
- descriptor ↔ DESCRIPTOR



BnInterface<I> 는 BBinder 와 I 를 동시에 상속 — addService 에 넘길 수 있고(BBinder), ledOn() 도 부를 수 있다(I).

▶ 애니메이션 1 · libbinder 계층 (객체 모델)

META 매크로와 interface_cast — asInterface 의 분기

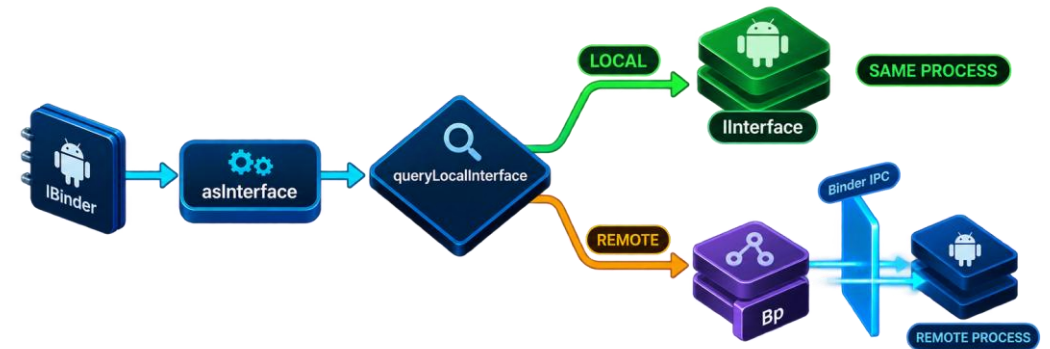
```
// ILedService.h
class ILedService : public IInterface {
    DECLARE_META_INTERFACE(LedService);    // descriptor, asInterface() 선언
    virtual void ledOn() = 0;
    enum { LED_ON = IBinder::FIRST_CALL_TRANSACTION };    // = 1
};
// ILedService.cpp
IMPLEMENT_META_INTERFACE(LedService, "android.my.ILedService");
```

```
interface_cast<ILedService>(binder) == ILedService::asInterface(binder)
```

binder->queryLocalInterface(descriptor) — "내 Process 의 BBinder 인가?"

로컬 → this 반환
Binder 안 거침 (Same process: YES)

원격 → new BpLedService(binder)
remote() = handle (Same process: NO)



- DECLARE 는 선언, IMPLEMENT 는 본문 + descriptor 문자열
- descriptor 가 service list 의 [...] 에 표시된다 (실습 4)
- AIDL 은 이 매크로 세트를 자동 생성 (실습 7 의 DO_NOT_DIRECTLY_USE_ME_ ...)
- 수작업 인터페이스는 #define DO_NOT_CHECK_MANUAL_BINDER_INTERFACE S 필요

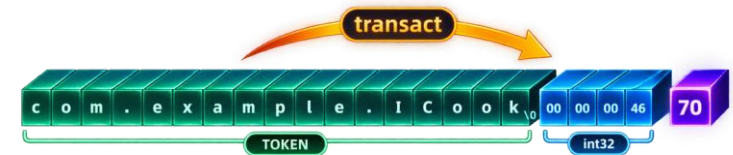
▶ 애니메이션 2 · interface_cast 와 META 매크로

Bn / Bp Transaction 흐름 — Parcel 은 순서가 계약

```
// Proxy (Client)
int BpLedService::ledOn(int ratio) {
    Parcel data, reply;
    data.writeInterfaceToken(ILedService::getInterfaceDescriptor());
    data.writeInt32(ratio);
    remote()->transact(LED_ON, data, &reply);
    return reply.readInt32();
}
```

```
// Stub (Server)
case LED_ON: {
    CHECK_INTERFACE(ILedService, data, reply); // 토큰 검사
    int ratio = data.readInt32(); // 쓴 순서대로
    reply->writeInt32(ledOn(ratio)); // 가상 호출 + 반환
    return NO_ERROR;
}
```

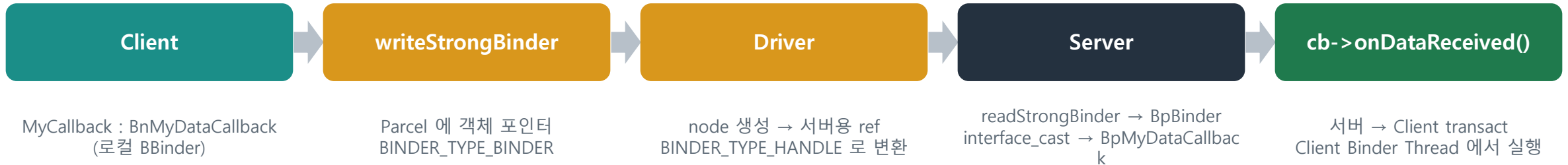
Parcel API	용도
writeInt32 / readInt32	기본형
writeString16 / readString16	문자열 (UTF-16)
writeStrongBinder / readStrongBinder	IBinder (Callback)
writeFileDescriptor / readFileDescriptor	FD (Pipe · ashmem)
writeInterfaceToken / CHECK_INTERFACE	인터페이스 검증



▶ 애니메이션 3 · Bn/Bp Transaction 흐름

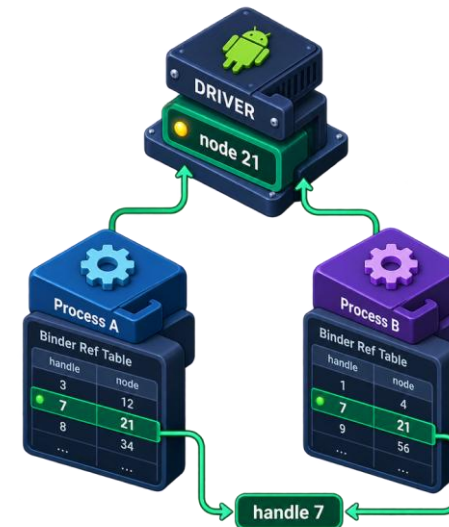
service call my.led_service 1 s16 "android.my.ILedService" i32 30 — 같은 바이트를 웹에서 손으로 만든 것

역방향 호출 — IBinder 가 Parcel 을 타면 node → handle



```

// Proxy
data.writeStrongBinder(IInterface::asBinder(cb));
// Stub
sp<IBinder> b = data.readStrongBinder();
sp<IMyDataCallback> cb = interface_cast<IMyDataCallback>(b);
cb->onDataReceived(String16("Hello")); // 역방향
// Client - 요청을 받아야 하므로
ProcessState::self()->startThreadPool();
IPCThreadState::self()->joinThreadPool();
  
```



- 포인터는 Process 를 넘지 않는다 — Driver 가 handle 로 변환
- Client 도 Server 역할 → Thread Pool 필수
- Java 는 Framework 가 자동, Native 는 직접
- RemoteCallbackList · linkToDeath 는 이 node/ref 위의 사망 통지

▶ 애니메이션 4 · Callback 역방향

최소 Native Binder Server — BBinder 하나 등록

목표 ProcessState · defaultManager · addService · joinThreadPool 다섯 줄로 서비스 등록 · 소요 20분

진행 순서

1. binder1/my_server.cpp 작성 (문서 그대로 5줄 골격)
2. `cd ~/android/day2 && mm → $OUT/system/bin/my_server_test_1`
3. WinSCP → `adb push /data → chmod 755`
4. 터미널 ① `/data/my_server_test_1` 실행
5. 터미널 ② `service list | grep my.service · binder_logs/proc/<PID> · service call my .service 1`

확인 포인트

- service list 에 my.service: [] (descriptor 없음)
- binder_logs 에 node 1개 · thread 1개
- service call → -1 (UNKNOWN_TRANSACTION)
- 서버 종료 후 service list 에서 사라짐



▶ 애니메이션 1 · libbinder 계층

Proxy 수작업 → Bn/Bp 대칭 완성

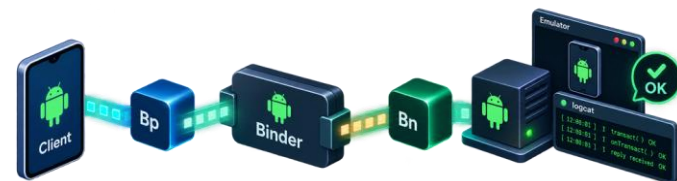
목표 ILedService · BpLedService · interface_cast (실습 3) → BnLedService::onTransact · LedService · descriptor (실습 4) · 소요 30 + 30분

진행 순서

1. 실습 3: ILedService.h (DECLARE_META_INTERFACE, LED_ON=1) · ILedService.cpp (BpLedService, IMPLEMENT_META_INTERFACE)
2. 실습 3: Server 는 raw BBinder 유지 → onTransact(code=1) 로 도착 확인
3. 실습 4: ILedService.h 에 BnLedService 선언 추가 · onTransact 구현 (CHECK_INTERFACE)
4. 실습 4: LedService : BnLedService 구현체 · startThreadPool 추가
5. 실습 4: service call 토큰 유무 비교 (s16 "android.my.ILedService")

확인 포인트

- 실습 3: pLed->ledOn() → 서버 onTransact(code=1)
- 실습 4: service list 에 [android.my.ILedService]
- 실습 4: 서버 로그 caller pid/uid
- 토큰 없으면 PERMISSION_DENIED



▶ 애니메이션 2 · interface_cast

▶ 애니메이션 3 · Bn/Bp Transaction 흐름

Parcel 인자·반환값 → Callback 역방향

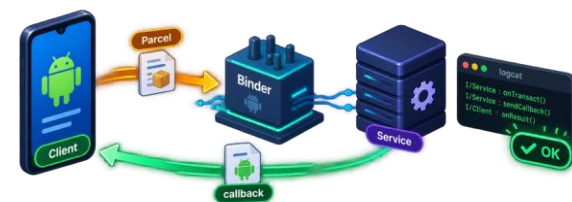
목표 writeInt32/readInt32 · reply (실습 5) → writeStrongBinder · Client Thread Pool (실습 6) · 소요 20 + 30분

진행 순서

1. 실습 5: binder3 복사 → ledOn(int ratio) → int 로 시그니처 변경
2. 실습 5: Proxy writeInt32 / Stub readInt32 · reply->writeInt32 / reply.readInt32
3. 실습 5: Step 6 — readInt32 앞에 readString16 을 끼워 순서 깨뜨리기
4. 실습 6: IMyDataCallback (Bn/Bp) + IMyDataService (Bn/Bp) 4파일
5. 실습 6: Client 가 MyCallback 을 넘기고 joinThreadPool · Step 5 — Thread Pool 제거 실험

확인 포인트

- ledOn(70)=70, ledOn(250)=100 (서버 보정)
- service call ... i32 30 → Parcel 0x1e
- 순서 깨면 값이 엉뚱함
- onDataReceived 가 Client 의 다른 tid 에서 실행



▶ 애니메이션 3 · Parcel 순서

▶ 애니메이션 4 · Callback 역방향

AIDL Native backend

25분 · aidl_interface · cpp / ndk backend · 생성 코드 · binder::Status · 안정성(unstable vs vintf)



aidl_interface — .aidl 에서 Bn / Bp / I 자동 생성

```
aidl_interface {
    name: "my_interface",
    srcs: ["binder7/IMyDataCallback.aidl", "binder7/IMyDataService.aidl"],
    local_include_dir: "binder7",
    backend: { cpp: { enabled: true } },      // ndk / java / rust
    unstable: true,                          // HAL: stability: "vintf" + ver
sions
}
cc_binary { name: "binder_server_7", srcs: ["binder7/server.cpp"],
    shared_libs: ["libbinder", "libutils"], static_libs: ["my_interfa
ce-cpp"] }
```



손으로 쓴 것 (실습 3~6)	생성 코드 (실습 7)
class BpLedService : BpInterface	BpMyDataService.h
BnLedService::onTransact + case LED_ON	BnMyDataService.h · TRANSACTION_registerCallback
writeInterfaceToken / CHECK_INTERFACE	_aidl_data.writeInterfaceToken · checkInterface(this)
IMPLEMENT_META_INTERFACE	DO_NOT_DIRECTLY_USE_ME_IMPLEMENT_META_INTERFACE
reply->writeInt32(ret)	Status 헤더 + _aidl_reply->writeInt32(_aidl_return)

- `aidl --lang=c++ -I . -o gen -h gen X.aidl` 로 직접 실행 가능 (실습 7 Step 5-1)
- `-cpp` / `-ndk` / `-java` 접미사 라이브러리
- 생성 Method: `Status f(in..., T* _aidl_return)`
- 실습 6 파일 4개 → .aidl 2개

▶ 애니메이션 5 · AIDL cpp backend 생성

binder::Status 와 NDK backend

```
// cpp backend - 생성 시그니처
Status add(int32_t a, int32_t b, int32_t* _aidl_return) override {
    if (a < 0) return Status::fromExceptionCode(Status::EX_ILLEGAL_ARGUMENT, "neg");
    *_aidl_return = a + b;
    return Status::ok();
}
// Client
Status st = svc->add(-1, 2, &r); // st.exceptionCode() == EX_ILLEGAL_ARGUMENT
```

	cpp backend (실습 7)	ndk backend (실습 10 HAL)
네임스페이스	android::	aidl::<pkg>::
라이브러리	libbinder	libbinder_ndk (vendor OK)
상태	binder::Status	ndk::ScopedAStatus
참조	sp<>	std::shared_ptr + SharedRefBase::make
등록 / 획득	addService / interface_cast	AServiceManager_addService / l::fromBinder

Status → Java 예외 매핑

- EX_SECURITY → SecurityException (실습 8-12)
- EX_ILLEGAL_ARGUMENT → IllegalArgumentException
- EX_ILLEGAL_STATE / EX_NULL_POINTER ...
- fromServiceSpecificError(n) → ServiceSpecificException
- Parcel 의 예외 헤더로 전달 → reply.readException()



▶ 애니메이션 5 · AIDL cpp backend (Status)

AIDL cpp backend 로 전환 — Bn/Bp 자동 생성

목표 실습 6 의 4파일을 .aidl 2개로 대체, 생성 코드 대조, aidl 직접 실행, Status 예외 · 소요 40분

진행 순서

1. binder7/IMyDataCallback.aidl · IMyDataService.aidl (add 추가)
2. server.cpp: BnMyDataService 상속, Status add(a, b, &ret) · EX_ILLEGAL_ARGUMENT
3. client.cpp: BnMyDataCallback 상속, add(40,2) / add(-1,2) 호출
4. mm → find out/soong/.intermediates/.../my_interface-cpp-source
5. Step 5-1: aidl --lang=cpp -I . -o gen -h gen → gen/ 8파일 · --lang=ndk 비교

확인 포인트

- add(40,2) → 42, add(-1,2) → EX_ILLEGAL_ARGUMENT
- 생성 코드에서 DO_NOT_DIRECTLY_USE_ME_IMPLEMENT_META_INTERFACE 찾기
- gen/ 에 Bn/Bp/I 8개 파일
- service list: [IMyDataService]



▶ 애니메이션 5 · AIDL cpp backend 생성

System Service · 보안 · SELinux

30분 · servicemanager 데몬 · getCallingUid · SystemServer 등록 경로 · ini
t.rc · SELinux avc · dumphsys



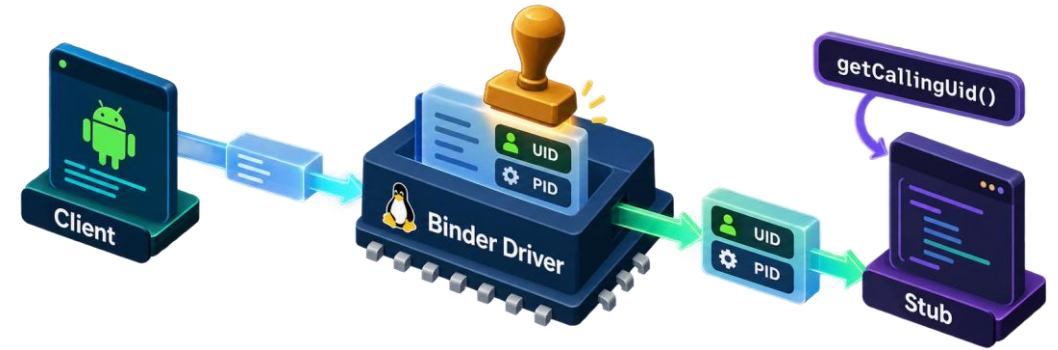
호출자 확인 — getCallingUid 는 커널이 보증한다

```

Status LEDON() override {
    uid_t uid = IPCThreadState::self()->getCallingUid();    // Driver
er 가 붙인 sender_euid
    pid_t pid = IPCThreadState::self()->getCallingPid();
    if (uid != AID_SYSTEM /*1000*/)
        return Status::fromExceptionCode(Status::EX_SECURITY, "Only
system UID");
    return Status::ok();
}

```

실행	uid	결과
/data/led_auth_client (root shell)	0	EX_SECURITY
su 1000 /data/led_auth_client	1000	LED ON
su 10050 ... / App (실습 12)	10050 / 10152	EX_SECURITY → SecurityException



- Driver 가 current->cred 에서 euid/pid 를 읽어 Transaction 에 부착 — 사용자 공간 위조 불가
- Parcel 에 "uid=1000" 이라 써도 서버는 data 가 아니라 Driver 값을 본다
- Stub Method(Binder Thread) 안에서만 유효 — main 에서 부르면 자기 uid
- Java checkCallingPermission() 도 이 uid 로 PackageManager 조회
- App 이 HAL 을 직접 못 부르는 이유 — system_server(1000) 만 허용

▶ 애니메이션 6 · UID 접근 제어

init.rc 로 데몬 등록 · SELinux avc 읽기

```
# /system/etc/init/rc_led_service.rc (Android.bp: init_rc:
["...rc"])
service rc_led_service /system/bin/rc_led_service
    class main
    user system
    group system
    disabled
    # start rc_led_service 로만 시작

$ start rc_led_service # = setprop ctl.start
$ getprop init.svc.rc_led_service # running
```

```
avc: denied { add } for pid=6301 comm="rc_led_service"
scontext=u:r:init:s0
tcontext=u:object_r:default_android_service:s0
tclass=service_manager permissive=1
```

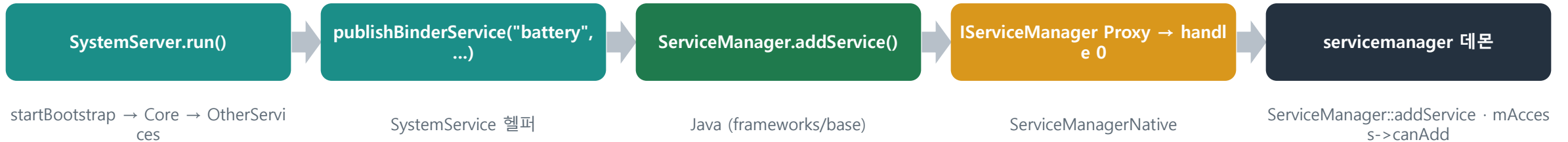
▶ [애니메이션 7 · init.rc 와 SELinux](#)

필드	의미
{ add } / { call }	거부된 권한
scontext	주체 도메인 — 전용 .te 없으면 init 도메인 그대로
tcontext	객체 라벨 (service_contexts 로 결정)
permissive=1 / 0	기록만 / 차단



- 필요한 세트: .te (domain + init_daemon_domain + binder_use + add_service) · service_contexts · file_contexts
- setenforce 1 → addService = -1 · setenforce 0 복귀
- audit2allow 로 거부 로그 → 규칙 초안

Java System Service 도 같은 servicemanager 에 도착한다



```

$ service list | grep -E "activity|car_service|led\.auth|vendor\.example"
activity: [android.app.IActivityManager]      ← Java, system_server
car_service: [android.car.ICar]                ← Java, com.android.car
led.auth: [ILedAuthService]                   ← C++ (실습 8)
vendor.example.led.ILed/default: [vendor.example.led.ILed] ← vendor HAL (실습 10)
  
```



- Binder(Java) ↔ BBinder(C++) ↔ AIBinder(NDK) — 같은 Driver 객체의 세 언어 표면
- `dumpsys <svc> = IBinder.dump() = DUMP_TRANSACTION` → C++ 에서 `dump()` 오버라이드하면 `dumpsys` 동작
- `servicemanager` 의 `canAdd/canFind` 가 SELinux 검사 지점 (실습 9 의 { add })

UID 기반 접근 제어 — `getCallingUid` · `EX_SECURITY` · `su 1000`

목표 AIDL cpp 서비스에 system UID 만 허용하는 검사를 넣고 세 UID 로 실행 비교 · 소요 30분

진행 순서

1. `auth/ILedAuthService.aidl` (`void LEDON()`)
2. `LedAuthService.cpp`: `getCallingUid != 1000` → `Status::fromExceptionCode(EX_SECURITY)`
3. `LedAuthClient.cpp`: `interface_cast` → `LEDON()` → `st.isOk` / `exceptionCode`
4. `mm` → `push` → 터미널 ① 서버 실행
5. 터미널 ②: `/data/led_auth_client · su 1000 /data/led_auth_client · su 10050 ...`

확인 포인트

- uid 0 → `EX_SECURITY`, uid 1000 → succeeded, 10050 → `EX_SECURITY`
- 서버 로그 uid/pid 가 Client 실제 값과 일치
- Step 6: Parcel 로 uid 위조 불가 이해



▶ 애니메이션 6 · UID 접근 제어

init.rc 로 Service 자동 시작 + SELinux 기초

목표 손으로 띄우던 데몬을 init 이 띄우게 하고, avc: denied 를 읽고, enforcing 에서 차단을 본다 · 소요 40분

진행 순서

1. rc_service/rc_led_service.cpp (ALOGI 로 로그) · rc_led_service.rc (user system, disabled)
2. mm → adb remount → push /system/bin, /system/etc/init → chcon → adb reboot
3. start rc_led_service → getprop init.svc.* → ps (PPID 1, uid system) → logcat -s rc_led_service
4. service list | grep rc.led · service call rc.led 1 · stop
5. Step 5: logcat | grep avc → scontext=u:r:init:s0 읽기 → setenforce 1 → addService=-1 → setenforce 0

확인 포인트

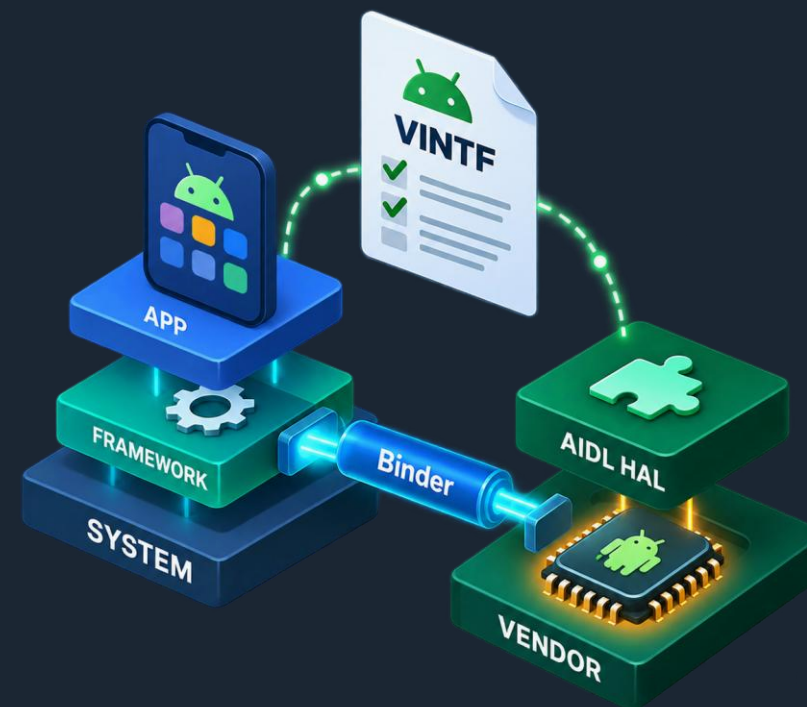
- start 후 running, PPID 1, uid system
- logcat 에 addService = 0
- avc: denied 의 scontext / tcontext / tclass 해석
- enforcing 에서 -1, permissive 복귀



▶ 애니메이션 7 · init.rc 와 SELinux

HAL — AIDL HAL · VINTF

30분 · Legacy → HIDL → AIDL HAL · system/vendor 경계 · 인스턴스 이름 규칙 · manifest ↔ compatibility matrix



AIDL HAL — system / vendor 경계와 4가지 규칙

system 파티션 — Framework · system_server · libbinder

LightsService (Java, uid 1000)
/ led_hal_client (실습 10)

compatibility_matrix.xml
"ILights v2 이상 필요"

/dev/binder — 같은 Driver (HIDL 시절 /dev/hwbinder)

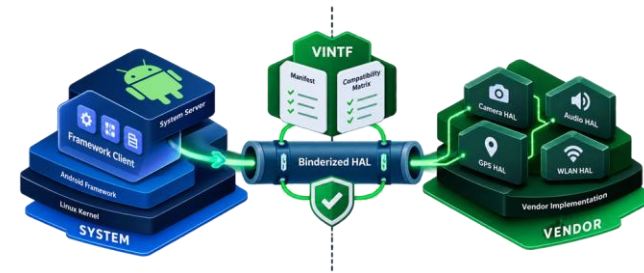
vendor 파티션 — SoC / 제조사 · libbinder_ndk 만

vendor.example.led-service
vendor: true · aidl::.....:BnLed

/vendor/etc/vintf/manifest.xml
"ILights v2 제공"

▶ [애니메이션 8 · AIDL HAL 과 VINTF](#)

규칙	내용
① 이름	<pkg>.<Interface>/<instance> — android.hardware.light.ILights/default
② 라이브러리	vendor 는 libbinder_ndk 만 → NDK backend (aidl::, ScopedAStatus)
③ 모듈	vendor: true → /vendor/bin · vendor_available 로 인터페이스 공유
④ 호출자	App 은 직접 못 부름 — system_server(uid 1000) 가 대신



- VINTF: manifest(제공) ↔ compat matrix(요구) 부팅 시 대조 → 불일치면 부팅 실패
- stability: "vintf" 면 manifest 없이 addService 거부 · 실습은 unstable: true 로 우회
- HAL 세대: Legacy .so → HIDL(hwbinder) → AIDL HAL (Android 11~, 16 기본)

AIDL HAL 구조 따라 만들기 — NDK backend · vendor · VINTF

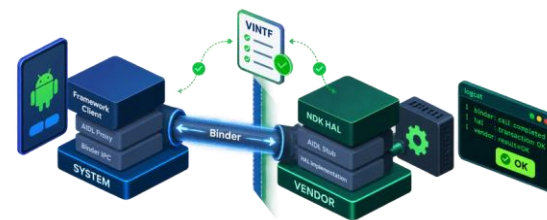
목표 실제 HAL 과 같은 형태(패키지명, ndk, vendor: true)로 최소 LED HAL 을 만들고 VINTF manifest 를 읽는다 · 소요 45분

진행 순서

1. led_hal/vendor/example/led/ILed.aidl (package vendor.example.led)
2. LedHal.cpp: aidl::vendor::example::led::BnLed · ScopedAStatus · AServiceManager _addService("...ILed/default")
3. LedHalClient.cpp: AServiceManager_checkService → ILed::fromBinder → setOn/is On
4. mm → \$OUT/vendor/bin (vendor: true) → push → 실행 on/off
5. Step 7: /vendor/etc/vintf/manifest.xml 에서 android.hardware.light 항목 읽기

확인 포인트

- service list: vendor.example.led.ILed/default
- \$OUT/vendor/bin 에 서비스, system/bin 에 client
- on/off 동작 + logcat
- manifest 의 <hal format="aidl"> 항목 해석



▶ 애니메이션 8 · AIDL HAL 과 VINTF

Java System Service 들여다보기 — SystemServer · ServiceManager · dumpsys

목표 system_server 의 Java Service 등록 경로를 소스에서 따라가고 오늘 만든 C++ 서비스와 같은 테이블에 있음을 확인 · 소요 30분

진행 순서

1. `grep startBootstrapServices / startCoreServices / startOtherServices — SystemServer.java`
2. `ServiceManager.addService → ServiceManagerNative → handle 0 → servicemanager 데몬 (:tag ServiceManager::addService)`
3. Emulator: service list 에서 Java · C++ · vendor HAL 나란히 확인
4. `dumpsys -l · dumpsys activity services · (선택) LedAuthService 에 dump() 추가 → dumpsys led.auth`
5. `BatteryService` 의 `publishBinderService · BinderService extends Binder` 따라가기

확인 포인트

- 3단계 함수와 `addService` 호출 위치
- Java·C++·HAL 이 한 service list
- `dumpsys led.auth` 출력 (선택)
- `mAccess->canAdd` 가 SELinux 검사 지점



App 에서 Native Service 호출 — transact 직접 · SecurityException (선택 / 과제)

목표 AIDL Java backend 없이 IBinder.transact 로 C++ 서비스를 부르고 App UID 로 거부되는 것을 본다 · 소요 30분

진행 순서

1. NativeCallLab 프로젝트 · 리플렉션으로 `android.os.ServiceManager.getService("led.auth")`
2. `binder.getInterfaceDescriptor()` → `ILedAuthService`
3. `Parcel: writeInterfaceToken("ILedAuthService")` → `transact(1)` → `reply.readException()`
4. 실습 8 서버 실행 중 앱 실행 → `SecurityException`
5. (과제) platform 키 서명 + `sharedUserId=android.uid.system` → 성공

확인 포인트

- descriptor 가 `ILedAuthService`
- `readException` → `SecurityException`
- 서버 로그 uid = App uid (10xxx)
- hidden API 라 리플렉션 필요



▶ 애니메이션 6 · UID 접근 제어

실습 13 (데모) · Binder 호출 비용 · strace 로 ioctl 관찰

```
$ strace -f -e trace=ioctl,mmap,openat /data/my_client_test_4 50 2>&1 | grep -E "bind
er|ioctl"
openat(AT_FDCWD, "/dev/binderfs/binder", O_RDWR|O_CLOEXEC) = 3      ← ProcessState::s
elf()
ioctl(3, BINDER_VERSION, ...) = 0
mmap(NULL, 1040384, PROT_READ, MAP_PRIVATE|MAP_NORESERVE, 3, 0)    ← 1 MB - 8 KB Bu
ffer
ioctl(3, BINDER_SET_MAX_THREADS, [15]) = 0                        ← Thread Pool 상한
ioctl(3, BINDER_WRITE_READ, ...) = 0                             ← checkService (h
andle 0)
ioctl(3, BINDER_WRITE_READ, ...) = 0                             ← ledOn() transac
t
```

관찰	의미
30~80 μ s / call (Emulator)	Binder 왕복 비용 — 루프 안에서 IPC 금지
openat + mmap 1 MB	ProcessState 생성 시 1회
BINDER_WRITE_READ	모든 transact() 의 실체
startThreadPool 없음 → tid 동일	main 하나가 순차 처리 · 있으면 tid 분산



Day 2 Worksheet — 20문제, 20분

유형	문제	범위
객관식	Q1 ~ Q8	ProcessState · BBinder · interface_cast · FIRST_CALL_TRANSACTION · writeStrongBinder · libbinder_ndk · permissive · start
단답형	Q9 ~ Q13	joinThreadPool · getCallingUid · 1000 · aidl_interface · servicemanager
빈칸 채우기	Q14 ~ Q17	META 매크로 · writeInterfaceToken/CHECK_INTERFACE · Status/_aidl_return · HAL 이름 규칙
순번 지정	Q18 ~ Q19	ledOn(70) 처리 순서 · init.rc 데몬 등록 순서
선잇기	Q20	C++ 클래스 ↔ Java 클래스

- Day2_Worksheet.html 을 열어 풀고 채점하기 → 문제별 해설 확인
- 종이로 풀 경우 인쇄 (문제만) → 강사가 정답·해설 모두 보기로 풀이
- 16 / 20 이상이면 오늘 내용을 충분히 이해한 것



▶ [Day 2 Worksheet 열기](#)

오늘 확인한 것

libbinder

ProcessState(Process) · IPCThreadState(Thread) · BBinder/BpBinder · BnInterface/BpInterface

수작업 Bn/Bp

META 매크로 · interface_cast 분기 · Parcel 은 순서가 계약 · IBinder 는 node→handle

AIDL

aidl_interface 가 Bn/Bp/I 생성 · Status 가 예외 · ndk backend 는 vendor 용

보안·시스템

getCallingUid 는 커널 보증 · init.rc 로 uid 고정 · SELinux 도메인×객체×권한

HAL

system/vendor 경계 · <pkg>.<l>/<inst> · libbinder_ndk · VINTF manifest ↔ matrix

